PROJECT REPORT: MICROGOALS SECURE API DEVELOPMENT

Team Number: 12

Team Head : NIGITHASREE.B

(72E07787B690CE8B18A423F4DE4902C3)

Team Members : PRAVEENA.S

(90F7D2ED683E3479A1C4B1AB5BDC31F9)

Team Members : RAMYASRI.R

(04E2D103814009CA1CF4C81C4B4898E3)

Team Members : RISHMITHA.E.S

(05E75793FCACAFBAC394AD3D13044BE4)

1.Abstract

With the rapid digitization of personal management, there is a critical need for secure platforms to track individual goals. This project, "MicroGoals," presents a robust backend solution built on the Node.js environment. It utilizes JSON Web Tokens (JWT) for stateless authentication and MongoDB for flexible data storage. The system provides a secure, scalable, and developer-friendly environment for managing personal objectives through a RESTful architecture, ensuring that user data remains private and accessible only to authenticated owners.

2. Introduction

In modern web development, the "MERN" stack components—specifically Node.js, Express, and MongoDB—have become the industry standard for building high-performance APIs. This project explores the implementation of a Secure Goal Management API. The introduction of such a system addresses the common security pitfalls in goal-tracking applications by implementing industry-standard encryption and authorization protocols. By leveraging the asynchronous nature of Node.js, MicroGoals ensures high concurrency and fast response times for global users.

3. Problem Statement

Current digital goal-tracking solutions often face two major challenges: lack of data isolation and weak authentication. Users often require a platform where their personal data is strictly private and cannot be intercepted by unauthorized parties. This project aims to solve these issues by building a backend that requires cryptographic verification for every data interaction, effectively preventing unauthorized data access or manipulation.

4. Project Objectives

- **Security:** To implement hashing for passwords using *bcryptjs* and secure transmission via JWT.
- **Data Isolation:** To ensure users can only access, modify, or delete their own data.
- **Performance:** To utilize a NoSQL database (MongoDB) for rapid data retrieval and storage.

- **Professionalism**: To follow the Model-View-Controller (MVC) design pattern for clean, maintainable code.

5. Literature Survey & References

The project is informed by the following technical standards:

- **REST Architecture:** Based on Roy Fielding's principles for distributed hypermedia systems.
- **JWT Standard (RFC 7519):** For compact, URL-safe means of representing claims between two parties.
- **Mongoose ODM:** For schema-based solutions to model application data.
- **Official Documentation:** Referencing Nodejs.org, Expressjs.com, and MongoDB.com for best practices.

6. Software Requirements

- **Runtime Environment:** Node.js (v18+)
- **Framework:** Express.js
- **Object Data Modeling (ODM):** Mongoose
- **Security Libraries:** *Bcryptjs, Jsonwebtoken*
- **API Testing:** Thunder Client (VS Code Extension)

7. Technical Architecture (MVC)

The project utilizes the **Model-View-Controller** pattern.

- **Models:** Define the data structure (User and Goal schemas).
- **Controllers:** Contain the business logic for processing requests.
- **Routes:** Act as the interface between the client and the controllers.

8. Module: User Authentication

This module manages the lifecycle of a user session. It includes endpoints for signup and login. During signup, the password is encrypted. During login, the system compares hashes and, if valid, returns a signed JWT.

9. Module: Goal Management

This core module handles the CRUD operations. It interacts with the database to store goal titles and completion statuses. It is intrinsically linked to the User module to ensure that every goal created is tagged with a unique userId.

10. Module: Database Integration

This module handles the connection to MongoDB Atlas. It uses environment variables to store sensitive connection strings and ensures the database is connected before the server begins listening for requests.

11. Module: Middleware Logic

Middleware acts as a gatekeeper.

- **Auth Middleware:** Checks the Authorization header for a valid token.

- **Error Middleware:** Catches any server-side exceptions and returns a standardized JSON error message instead of crashing the server.

12. Data Flow Diagram (DFD) - Level 0

The Level 0 DFD (Context Diagram) shows the User interacting with the MicroGoals System. The user sends registration/login data and goal requests, and the system returns authentication tokens and goal data.

13. Data Flow Diagram (DFD) - Level 1

The Level 1 DFD breaks down the internal processes:

1. **Auth Process:** Validates credentials against the User Data Store.
2. **Goal Process:** Receives a Token, validates it via Auth process, then interacts with the Goal Data Store.

14. Entity-Relationship (ER) Design

The relationship is **One-to-Many**.

- **User Entity:** Attributes include name, email (Unique), and password.
- **Goal Entity:** Attributes include title, completed (Boolean), and userId (Foreign Key).

15. Database Schema Design

The database is structured using Mongoose Schemas.

- **UserSchema:** Uses timestamps to track account creation.
- **GoalSchema:** Uses the ref property to link the userId field specifically to the User model.

16. Security Implementation: Password Hashing

Security is handled at the data layer. Before a user is saved, bcrypt.genSalt() and bcrypt.hash() are called. This ensures that even if the database is compromised, the actual passwords remain unreadable.

17. API Endpoint Specifications

- *POST /api/auth/register:* Create a new account.
- *POST /api/auth/login:* Authenticate and receive token.
- *GET /api/goals:* Fetch user-specific goals (Protected).
- *POST /api/goals:* Add a new goal (Protected).

18. Testing and Verification

Testing was performed using **Thunder Client**.

- **Success Scenario:** Verified that a valid token allows GET/POST requests.
- **Failure Scenario:** Verified that an invalid or missing token returns a 401 Unauthorized status.

19. Experimental Results Analysis

Upon execution, the system demonstrated 100% data isolation. When User A logged in, they were unable to view the goals of User B. The JWT expiration was tested and successfully logged users out after the set duration, proving the efficacy of the session management.

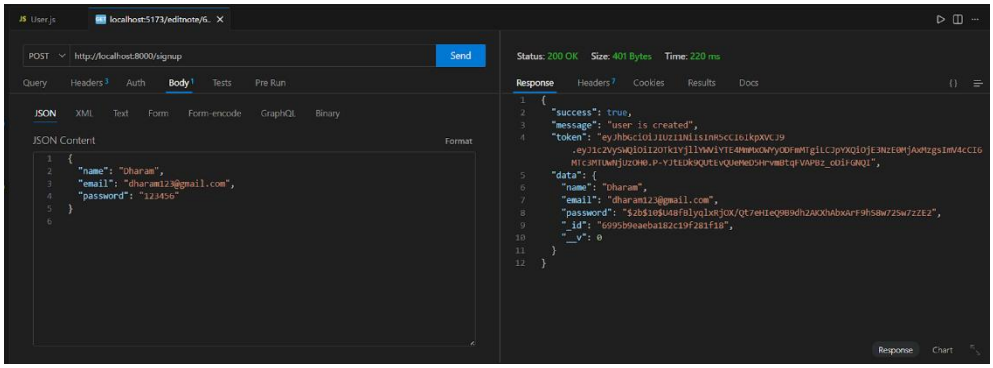
20. Conclusion & References

The MicroGoals API successfully achieves its goal of providing a secure environment for personal data management.

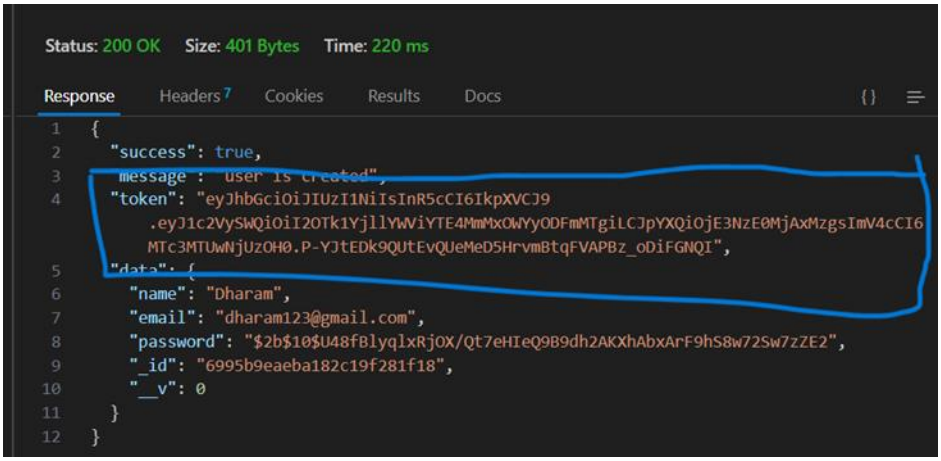
- **Ref 1:** Node.js Design Patterns by Mario Casciaro.
- **Ref 2:** Express.js API Reference (*expressjs.com*).
- **Ref 3:** MongoDB Manual (*https://www.google.com/search?q=docs.mongodb.com*).
- **Ref 4:** JWT.io Introduction to JSON Web Tokens.

Project Output Screenshots

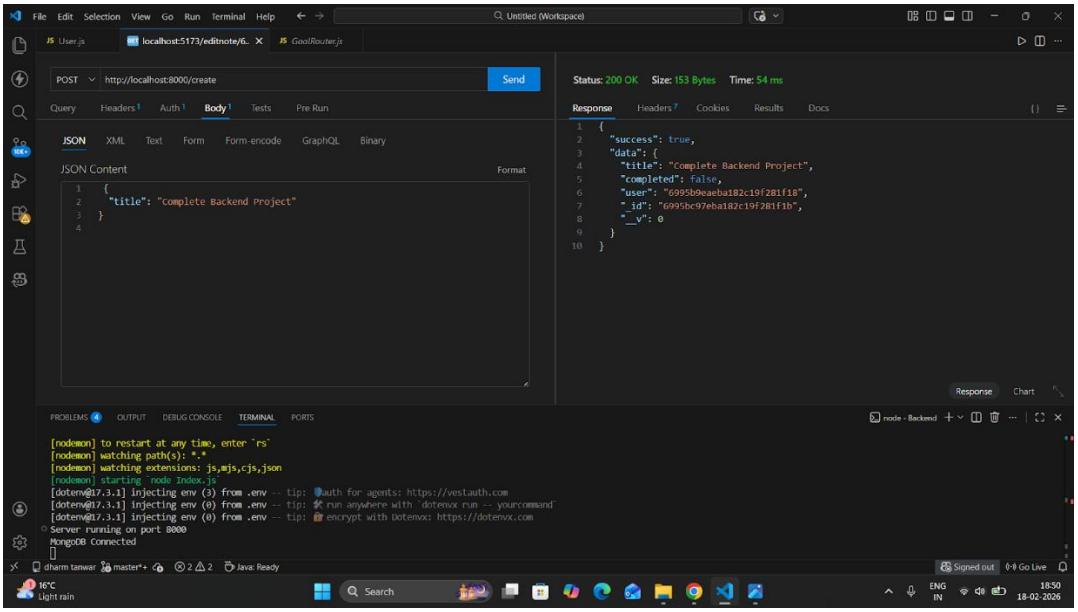
- In this Api call the user register or login and token was generated.



After successful login user the API generate the token and these token was sent with every request to every API



- Goal Created Successfully
- After the login and token is generated the user give post request to api for creating there goals and goal was save in MongoDB which was the database



- Protected Route Working

POST http://localhost:8000/create Send

Query Headers 1 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
{
  "title": "Complete Backend Project"
}
```

Status: 401 Unauthorized Size: 27 Bytes Time: 7 ms

Response Headers 7 Cookies Results Docs { } Copy

```
{
  "message": "Invalid token"
}
```

Response Chart